

백엔드 성능 + ML 포트폴리오 플랜

백엔드 성능 + ML 포트폴리오 플랜

목적

산업 안전 멀티모달 시스템의 백엔드 / 시스템 성능 최적화 를 메인으로, ML 추론 최적화 + Warning F1 회복 을 보너스 트랙으로 진행해 백엔드 취업용 포트폴리오를 완성한다.

핵심 원칙

- 측정 → 개선 → 비교 → 시각화 순서 엄격 준수
- 정량 비교 자료 (Before/After) 매 단계 확보
- 백엔드 파이프라인 성능 (latency, fps) 메인, ML 정확도 (F1) 별도 트랙
- 시간이 아니라 단계 완료 기준으로 다음 진행

전체 흐름

- Phase 1 – BE-1: 측정 + 문제 정의
↓ (병목 정량 데이터 확보)
- Phase 2 – BE-2: 백엔드 툴 도입 + 비교
↓ (Phase 1 비교)
- Phase 3 – BE-5: 모니터링 (Prometheus/Grafana)
↓ (시각 자료 확보)
- Phase 4 – BE-3: 모델 추론 최적화 (ONNX/quantization)
↓ ("추가로 더 깊이 파봤다")
- Phase 5 – ML 트랙: Warning F1 회복 + 다량 데이터 학습
↓
- Phase 6 – Polish: README + 코드 정리 + 면접 준비

Phase 1 — BE-1: 측정 + 문제 정의

목적

“서버/파이프라인의 어디가 느린가?” 를 객관적 데이터로 답한다. 이후 모든 페이지의 비교 기준 (Baseline) 이 됨.

다루는 것

- 파이프라인 단계별 latency (frame read / detect / fusion / publish)
- 5Hz frame rate 안정성
- HTTP 엔드포인트 응답 시간
- WebSocket 처리량

다루지 않는 것

- 모델 정확도 (F1, accuracy) — Phase 5 에서
- 모델 구조 변경 — Phase 5 에서
- 새 기능 추가 — 이 페이지는 측정만

작업 단계

Step 1.1 — Pipeline timing 계측 코드 추가

- `realtime_camera.py` 메인 루프에 단계별 `time.perf_counter` 측정
- 각 단계 (`frame_read`, `detect`, `fusion`, `publish`) timestamp 기록
- 결과를 JSON Lines 로 누적 (`metrics/pipeline.jsonl`)

Step 1.2 — FastAPI middleware 로 HTTP latency 자동 측정

- 모든 엔드포인트 진입/퇴장 시간 측정
- `metrics/http.jsonl` 로 누적
- `request.url.path`, `method`, `status_code`, `duration_ms` 기록

Step 1.3 — Locust 부하 테스트 (웹 패턴 엔드포인트만)

- 대상: `/audio/score`, `/incident-logs`, `/send-alert`, `/reports/generate`
- `locustfile.py` 작성
- 시나리오: 동시 사용자 100~500, 30분간

Step 1.4 — 분석 + 시각화

- JSON Lines 로그 → `pandas` / `matplotlib`
- 히스토그램 (각 단계 시간 분포)
- 시계열 (시간대별 변화)
- 백분위 그래프 (p50, p95, p99)

Step 1.5 — Baseline 문서화

- `docs/baseline/BASELINE.md` 작성
- 측정 환경, 측정 방법, 결과 표
- 식별된 병목 우선순위 명시
- 그래프 임베드

산출물

- `metrics/` 디렉터리 (JSON Lines 로그)
- `docs/baseline/*.png` 그래프 (5~6장)
- `docs/baseline/BASELINE.md` 문서
- `tools/locustfile.py`
- 식별된 병목 1, 2, 3 우선순위 (Phase 2 작업 결정 근거)

Phase 2 — BE-2: 백엔드 툴 도입 + 비교

목적

Phase 1 에서 식별한 병목을 백엔드 기술 도입으로 해결. **모든 도입 후 즉시 재측정** 해서 Phase 1 과 비교.

다루는 것

- DB 최적화 (인덱스, N+1, connection pool)
- Redis 캐싱
- 비동기 처리 (큐, 메시지 broker)
- 동기 → 비동기 전환

다루지 않는 것

- 모델 자체 변경 — Phase 4 / 5
- 모니터링 도구 — Phase 3
- 인프라 확장 (다중 인스턴스 등) — 우선순위 낮음

작업 단계

(Phase 1 결과의 병목 우선순위에 따라 일부 선택)

Step 2.1 — PostgreSQL 인덱스 추가

- 느린 쿼리 식별 (EXPLAIN ANALYZE)
- 추가 후보:
 - incident_logs(created_at) — 시계열 조회
 - incident_logs(maker_id) — 필터링
 - reports(date) — 일자 조회

Step 2.2 — N+1 쿼리 해결

- SQLAlchemy selectinload, joinedload 적용
- 의심 지점: /incident-logs, /reports/{id}/html

Step 2.3 — Connection pool 튜닝

- pool_size, max_overflow, pool_pre_ping 설정
- 부하 테스트로 최적값 찾기

Step 2.4 — Redis 도입

- docker-compose.yml 에 Redis 서비스 추가
- aioredis 또는 redis-py 설치
- 캐시 대상:
 - /audio/score (0.5초 TTL — 폴링 부담 감소)
 - 자주 조회되는 incident_logs

Step 2.5 — 비동기 작업 큐 도입

- 도구: Celery (또는 ARQ — 가벼운 대안)
- 비동기화 대상:
 - LLM 보고서 생성 (/reports/generate — 수십 초 블로킹 → 즉시 응답)
 - 무거운 이미지 처리

Step 2.6 — Redis Streams 또는 Kafka (메시지 큐)

- 현재 MQTT 만 있음 → 영속 메시지 큐 추가
- 대안:
 - Redis Streams (이미 있는 Redis 활용, 가벼움)
 - Kafka (full-scale, 복잡도 높음)
- 적용 영역:
 - Fusion subprocess → Backend 알림 전송
 - DB 저장 비동기화 보강

Step 2.7 — 각 도입 후 재측정

- 각 Step 끝나면 Phase 1 의 측정 재실행
- BASELINE 표에 추가 (Before / After / 개선 %)

산출물

- docs/baseline/BASELINE.md 갱신 (단계별 누적 비교)
- 각 Step 의 코드 변경 (commit 단위 기록)
- 추가 인프라 (Redis, Celery worker 등) 도커 컴포즈 통합

Phase 3 — BE-5: 모니터링 (Prometheus + Grafana)

목적

실시간 가시성 확보 + 포트폴리오용 시각 자료 생성. 스크린샷 1장이 백 마디.

다루는 것

- Prometheus 메트릭 수집
- Grafana 대시보드 시각화
- API / DB / Redis 메트릭 노출

다루지 않는 것

- 알람 / 알림 시스템 — 시간 남으면 보너스
- 분산 추적 (Jaeger 등) — 우선순위 낮음
- 로그 집계 (ELK/Loki) — 보너스

작업 단계

Step 3.1 — Prometheus 추가

- docker-compose.yml 에 prometheus 서비스 추가
- docker/prometheus.yml 설정 작성
- scrape target 설정 (backend, postgres, redis)

Step 3.2 — Grafana 추가

- docker-compose.yml 에 grafana 서비스
- 데이터 소스 (Prometheus) 자동 연결
- admin 로그인 설정

Step 3.3 — FastAPI 메트릭 노출

- prometheus-fastapi-instrumentator 설치
- /metrics 엔드포인트 자동 노출
- 기본 메트릭: request rate, latency 분포, status code

Step 3.4 — DB / Redis exporter 추가

- postgres_exporter 컨테이너
- redis_exporter 컨테이너
- Prometheus 가 둘 다 scrape

Step 3.5 — Pipeline 메트릭 커스텀

- prometheus_client 로 파이프라인 단계별 메트릭 노출
- Histogram: pipeline_step_duration_seconds
- Counter: frames_processed_total, frames_dropped_total
- Gauge: current_workers_tracked, audio_score

Step 3.6 — Grafana 대시보드 디자인

- 패널 6~8개:
 - End-to-end latency (시계열 + p99)
 - Pipeline 단계별 latency 분포
 - Frame rate 안정성
 - HTTP 엔드포인트 RPS / latency
 - DB 연결 / 쿼리 시간
 - Redis hit rate
 - 메모리 / CPU
 - 사고 발생률 (incident_logs)

Step 3.7 — 스크린샷 + 문서화

- 정상 동작 시 대시보드 스크린샷
- 부하 테스트 중 대시보드 스크린샷
- docs/monitoring/ 디렉터리에 정리

산출물

- docker-compose.yml 에 prometheus + grafana + exporters 통합

- docker/prometheus.yml, docker/grafana/dashboards/*.json
 - docs/monitoring/dashboard_*.png 스크린샷
 - docs/monitoring/MONITORING.md 설명서
-

Phase 4 — BE-3: 모델 추론 최적화

목적

백엔드 일반 기술로 줄일 수 있는 latency 다 줄였으니, 남은 병목 = 모델 추론 시간 을 ONNX / quantization 으로 추가 단축. “백엔드 깊이 + MLOps 영역까지” 어필.

다루는 것

- PyTorch → ONNX 변환
- INT8 quantization
- ONNX Runtime 추론
- 정확도 손실 측정

다루지 않는 것

- 모델 구조 변경 — Phase 5
- 학습 데이터 변경 — Phase 5
- 정확도 향상 — Phase 5

작업 단계

Step 4.1 — Fusion 모델 ONNX 변환

- torch.onnx.export 로 변환
- 변환 검증 (입출력 shape 일치)
- ONNX Runtime 으로 로드 + 추론 가능 확인

Step 4.2 — Fusion latency 비교

- PyTorch FP32 vs ONNX FP32
- 1000 프레임 평균 측정
- BASELINE.md 에 추가

Step 4.3 — Fusion INT8 quantization

- ONNX Runtime 의 quantization tool 사용
- 동적/정적 quantization 둘 다 시도
- 정확도 손실 측정 (analyze_fusion.py 재실행)
- latency 비교

Step 4.4 — YOLO ONNX 변환 (선택)

- Ultralytics 가 ONNX export 지원

- 변환 후 latency 비교
- (정확도 손실은 거의 없을 것으로 예상)

Step 4.5 — RealtimeInference 통합

- model/fusion/runtime/realtime_camera.py 가 ONNX 모델 사용 옵션
- 환경변수 INFERENCE_BACKEND=pytorch | onnx 로 전환
- 운영에서 ONNX 사용

Step 4.6 — 통합 부하 테스트

- 모든 최적화 (Phase 1~4) 적용 후 final 측정
- BASELINE 의 모든 항목 갱신

산출물

- model/fusion/checkpoints/best_*.onnx
- tools/convert_to_onnx.py
- BASELINE.md 의 PyTorch / ONNX FP32 / ONNX INT8 비교표
- 정확도 손실 보고서

Phase 5 — ML 트랙: Warning F1 회복 + 다량 데이터 학습

목적

Warning 클래스 F1 (현재 forklift 0.60 / dropzone 0.71) 을 회복. **ML 측면 깊이 어필** + 정량 비교표 추가.

다루는 것

- 임계값 튜닝
- 클래스 가중치 (loss weighting)
- 합성 시나리오 확장
- Data augmentation
- 재학습 + 비교

다루지 않는 것

- Phase 4 의 ONNX 와 별개 (정확도 트랙)
- 백엔드 인프라 변경

작업 단계

Step 5.1 — 임계값 튜닝

- analyze_fusion.py 의 PR 곡선 보고 적정점 찾기
- THRESH_WARN, THRESH_DANGER 조정
- F1 변화 측정

Step 5.2 — 클래스 가중치 추가

- BCEOnProb에 pos_weight 적용
- warn 클래스 가중치 ↑
- 재학습 후 비교

Step 5.3 — 합성 시나리오 확장

- scenarios_synthetic.py의 24개 → 100+
- 특히 warn 영역 시나리오 보강 (모호한 위험 케이스)
- analyze_fusion.py의 misclassified.txt 참고

Step 5.4 — Data Augmentation

- dataset.py에 augmentation 추가:
 - 위치에 가우시안 노이즈 (작은 분포)
 - 시간 axis shift (window 시작점 랜덤)
 - 미러링 (X 축)
- 재학습 + 비교

Step 5.5 — 재학습 + 비교표

- 각 단계별 best_forklift.pt / best_dropzone.pt 갱신
- analyze_fusion.py 결과 비교
- 누적 비교표:
 - Baseline / + Threshold / + Weighting / + Scenarios / + Augmentation

산출물

- 새 best_*.pt
- docs/ml/ML_IMPROVEMENT.md (단계별 F1 비교)
- docs/ml/confusion_matrix_*.png (단계별)

Phase 6 — Polish: 포트폴리오 완성

목적

지금까지 만든 자료를 **GitHub README + 면접용 자료**로 정리.

작업 단계

Step 6.1 — README 재작성

- 한 줄 요약 + 데모 영상/GIF (있으면)
- 시스템 아키텍처 다이어그램
- 백엔드 성능 비교표 (메인 어필)
- 모니터링 대시보드 스크린샷 (시각 임팩트)
- 모델 추론 최적화 비교 (ONNX / quantization)

- ML F1 항상 비교 (보너스 어필)
- 기술 스택
- 실행 방법

Step 6.2 — 추가 문서

- docs/ARCHITECTURE.md — 시스템 전체 아키텍처
- docs/BENCHMARK.md — 모든 성능 측정 한 곳에
- docs/ML.md — 모델 설계 / 학습 / 평가 정리

Step 6.3 — 코드 정리

- 핵심 파일 docstring 점검
- 미사용 코드 제거
- 주석 한글 / 영어 일관성
- 디렉터리 구조 README 에 트리로 명시

Step 6.4 — 면접 준비

- “이 프로젝트에서 가장 어려웠던 점” 스토리
- “어떻게 병목 식별했나” 답변
- “왜 Redis 가 아니라 X 였나” 같은 의사결정 근거
- 시스템 다이어그램 직접 그리며 설명 연습

Step 6.5 — 외부 노출 (선택)

- LinkedIn 글 작성
- dev.to / Medium 블로그
- 데모 영상 (1~2분)

산출물

- 폴리시된 GitHub 저장소
- README + 추가 markdown 문서들
- 면접용 스토리 노트

페이지 간 연결 — 항상 비교 데이터 갱신

각 페이지 끝날 때마다 docs/baseline/BASELINE.md 의 비교표 갱신:

Phase	end-to-end latency	DB query	API p99	Frame stability
Baseline (Phase 1)	250ms	230ms	380ms	92%
+ DB 인덱스 (Phase 2)	230ms	25ms	280ms	92%
+ Redis 캐시 (Phase 2)	220ms	15ms	200ms	92%
+ Celery 비동기 (Phase 2)	200ms	15ms	50ms	95%
+ ONNX 변환 (Phase 4)	80ms	15ms	50ms	99%

→ 이 표 1개가 면접 시 가장 강력한 도구.

위험 / 대응

위험 1 — Phase 1 측정에서 큰 병목 안 발견

대응: 작은 병목 여러 개라도 다 정리해서 표 만들기. “이미 잘 짜여있었다” 도 가치 있음.

위험 2 — Phase 2 도입한 도구가 효과 미미

대응: 솔직히 비교표에 적기. “도입했지만 효과 X — 이미 충분했음” 도 학습.

위험 3 — Phase 4 ONNX 변환 실패 / 정확도 큰 손실

대응: PyTorch FP32 그대로 운영 + “시도 후 trade-off 발견” 의 솔직한 보고.

위험 4 — Phase 5 ML 향상이 미미

대응: “24개 합성 데이터의 한계” 솔직히 명시. 시도 자체가 ML 사고 어필.

위험 5 — Phase 6 시간 부족

대응: README 만든 무조건. 추가 문서는 우선순위 낮춰도 됨.

성공 기준 (Definition of Done)

각 페이지가 끝났다고 선언할 수 있는 기준:

- **Phase 1:** BASELINE.md + 그래프 5+ 장 + 식별된 병목 명시
- **Phase 2:** 도입한 각 도구의 Before/After 비교 데이터
- **Phase 3:** Grafana 대시보드 스크린샷 + 메트릭 노출 코드
- **Phase 4:** ONNX 변환 + latency 비교 + 정확도 손실 측정
- **Phase 5:** 새 best_*.pt + F1 비교표
- **Phase 6:** README polish + 면접 시연 가능한 상태

다음 행동

Phase 1 Step 1.1 부터 시작: - realtime_camera.py 메인 루프에 timing 측정 코드 추가 -
metrics/pipeline.jsonl 로 누적 시작